

Convenciones de Nomenclatura (Naming Conventions)

1. 🎯 Objetivo: ¿Qué son y por qué las usamos?

Las **Convenciones de Nomenclatura** (o *Naming Conventions*) son un conjunto de reglas unificadas que acordamos seguir para nombrar todos los elementos de un proyecto: archivos, variables, funciones, tablas de bases de datos, etc.

El objetivo principal es que el código sea profesional y fácil de entender por cualquier miembro del equipo (o por nuestro "yo" del futuro).

Las usamos para mejorar:

- **Claridad y Legibilidad:** Permiten comprender la estructura del proyecto de un solo vistazo.
- **Mantenibilidad:** Facilitan enormemente la depuración de errores y la actualización del código.
- **Coherencia y Colaboración:** Son esenciales para el trabajo en equipo, asegurando que todos los miembros sigan las mismas normas.

2. 🌐 Principios Generales del Nombre

Antes de las reglas, aplicamos estos principios al "pensar" un nombre:

- **Regla de Oro del Idioma:**
 - **Inglés:** Para todo el **código** (variables, funciones, tablas, rutas API, claves JSON).
 - **Español:** Para **comentarios**, **mensajes en consola** e **interfaces de usuario**.
- **Claridad sobre Brevedad:** Un nombre debe ser expresivo y descriptivo.
 - **Mal:** `const f = '...';`
 - **Bien:** `const defaultApiUrl = '...';`
- **Evitar Nombres Genéricos:** No debemos usar nombres ambiguos como `data`, `temp`, `var1` o `info`.
- **Verbos y Sustantivos:**
 - **Funciones/Métodos:** Usar verbos que describan la acción (ej. `getUser()`, `calculateTotal()`).
 - **Variables/Clases:** Usar sustantivos que describan la "cosa" (ej. `userList`, `Invoice`).

3. Reglas de Nomenclatura por Contexto

Las reglas cambian según el "contexto" o el lenguaje en el que estemos trabajando.

3.1. Contexto: Código (JavaScript y PHP)

- **Variables y Funciones:** Usamos **camelCase**. La primera palabra va en minúscula y las siguientes empiezan en mayúscula.
 - Ejemplos: `userList`, `validateForm()`.
 - **Aclaración `const`:** Esto **incluye** funciones, objetos o arrays declarados con `const` (Ej: `const showTypes = () => {...}`).
- **Clases y Constructores:** Usamos **PascalCase** (o **UpperCamelCase**). Todas las palabras comienzan en mayúscula.
 - Ejemplos: `UserModel`, `SessionController`.
- **Constantes (Valores Primitivos):** Usamos **SCREAMING_SNAKE_CASE**. Todo en mayúsculas, separado por guiones bajos.
 - **Aclaración:** Se reserva para `const` que almacenan valores fijos y primitivos (configuración, números mágicos, etc.).
 - Ejemplos: `MAX_USERS`, `API_URL`.

3.2. Contexto: Web y Archivos (HTML/CSS)

- **Archivos y Carpetas:** Usamos **kebab-case**. Todo en minúsculas, separado por guiones medios.
 - Ejemplos: `login-controller.js`, `user-form.css`, `conexion-jsphp/`.
- **Selectores CSS (IDs y Clases):** Usamos **kebab-case**. Permite una lectura clara en las hojas de estilo.
 - Ejemplos: `.main-container`, `#user-form-section`.
- **Etiquetas HTML:** Usamos **lowercase** (minúsculas), siguiendo el estándar W3C.
 - Ejemplos: `<div>`, `<section>`, `<input>`.

3.3. Contexto: API y Entorno

- **Rutas/Endpoints API:** Usamos **lowercase** y **sustantivos en plural** (estilo RESTful).
 - Ejemplos: `/users`, `/products`, `/users/{id}/posts`.
- **Claves JSON:** Usamos **camelCase**. La API debe devolver las claves en el mismo formato que usa el cliente (JavaScript).
 - Ejemplo: `{"userId": 1, "userName": "Javi"}`.
- **Variables de Entorno (`.env`):** Usamos **SCREAMING_SNAKE_CASE**.

- Ejemplos: `DB_HOST` , `DB_USER` , `API_KEY` .

3.4. Contexto: Bases de Datos (El Proceso de 3 Fases)

El diseño de BBDD tiene 3 fases, cada una con sus convenciones:

Fase 1: Modelo Conceptual (E/R)

Este es el "dibujo" inicial para entender el problema.

- **Entidades:** `MAYÚSCULAS` y `SINGULAR` (Ej: `USUARIO` , `CURSO`).
- **Atributos:** `minúsculas` (Ej: `nombre` , `email`).
- **Relaciones:** `minúsculas` (preferiblemente verbos, ej: `asiste_a` , `imparte`).
- **Notación PK:** Atributo `subrayado` o con círculo relleno.

Fase 2: Modelo Lógico (Relacional / Grafo)

Este es el "paso a tablas" en papel o en una herramienta, antes del SQL.

- **Tablas (Relaciones):** `MAYÚSCULAS` (Ej: `USUARIO` , `CURSO`).
- **Notación PK:** Atributo `subrayado` .
- **Notación FK (Clave Ajena):** Atributo en `cursiva` .
- **Notación Unique (Clave Alternativa):** Atributo en `negrita` .
- **Notación Nulos:** Atributo con `*` (asterisco) si permite nulos.

Fase 3: Modelo Físico (Código SQL)

Esta es la implementación final(`CREATE TABLE`).

- **Tablas:** `plural` y `snake_case` (minúsculas con guion bajo)(Ej: `usuarios`).
- **Columnas:** `singular` y `snake_case` (Ej: `fecha_creacion`).
- **Columna (PK):** `id` .
- **Columna (FK):** `[tabla_padre_singular]_id` (Ej: `usuario_id`).
- **Tablas Intermedias (N:M):** `[tablaA_plural]_[tablaB_plural]` (Ej: `usuarios_rols`).
- **Restricción (Constraint) PK:** `pk_[nombre_tabla]` (Ej: `CONSTRAINT pk_usuarios ...`).
- **Restricción (Constraint) FK:** `fk_[tabla_hija]_[tabla_padre]` (Ej: `CONSTRAINT fk_publicaciones_usuarios ...`).
- **Restricción (Constraint) Unique:** `uq_[tabla]_[columna]` (Ej: `CONSTRAINT uq_usuarios_email ...`).
- **Índices (Indexes):** `idx_[tabla]_[columna]` (Ej: `CREATE INDEX idx_usuarios_nombre ...`).
- **Vistas (Views):** `vw_[proposito]` (Ej: `CREATE VIEW vw_usuarios_activos ...`).

- **Procedimientos Almacenados:** `sp_[verbo]_[entidad]` (Ej: `CREATE PROCEDURE sp_insertar_usuario ...`).
- **Triggers:** `tr_[tabla]_[momento]_[evento]` (Ej: `tr_usuarios_after_insert`).

4. ⚡ Tabla Resumen (Cheatsheet Definitivo)

Contexto	Tipo de elemento	Convención	Ejemplo
CÓDIGO			
JS/PHP	Variables y Funciones (incl. <code>const</code> func/obj)	<code>camelCase</code>	<code>userList</code> , <code>const showTypes = ...</code>
JS/PHP	Clases y Constructores	<code>PascalCase</code>	<code>UserModel</code>
JS/PHP	Constantes (Valores primitivos/fijos)	<code>SCREAMING_SNAKE_CASE</code>	<code>MAX_USERS</code> , <code>API_URL</code>
<code>.env</code>	Variables de Entorno	<code>SCREAMING_SNAKE_CASE</code>	<code>DB_HOST</code> , <code>DB_PASSWORD</code>
WEB Y API			
Archivos	Carpetas y Archivos	<code>kebab-case</code>	<code>login-controller.js</code>
CSS	IDs y Clases	<code>kebab-case</code>	<code>.main-container</code>
HTML	Etiquetas	<code>lowercase</code>	<code><div></code> , <code><section></code>
API	Rutas / Endpoints	<code>lowercase (plural)</code>	<code>/api/users</code> , <code>/api/products</code>
API	Claves JSON	<code>camelCase</code>	<code>{"userId": 1, "userName": "Javi"}</code>
BBDD (Diseño Conceptual E/R)			
E/R	Entidades	<code>MAYÚSCULAS (Singular)</code>	<code>USUARIO</code> , <code>PUBLICACION</code>
E/R	Atributos	<code>minúsculas</code>	<code>nombre</code> , <code>email</code>
E/R	Relaciones	<code>minúsculas</code>	<code>tiene</code> , <code>asiste_a</code>
BBDD (Diseño Lógico / Grafo)			
Lógico	Notación PK	<code><u>subrayado</u></code>	<code>PERSONA (<u>DNI</u>, nombre)</code>
Lógico	Notación FK	<code>*cursiva*</code>	<code>VIVIENDA (... , *código_mun*)</code>

Contexto	Tipo de elemento	Convención	Ejemplo
Lógico	Notación Unique	**negrita**	PERSONA (DNI, **email**)
Lógico	Notación Nulos	* (asterisco)	PERSONA (DNI, nombre, telefono*)
BBDD (Implementación Física SQL)			
SQL	Tablas	snake_case (plural)	usuarios , publicaciones
SQL	Columnas (Generales)	snake_case (singular)	nombre , fecha_creacion
SQL	Columna (Clave Primaria)	id	id (en usuarios)
SQL	Columna (Clave Foránea)	[padre_singular] _id	usuario_id (en publicaciones)
SQL	Tabla Intermedia (N:M)	[tablaA]_[tablaB] (plural)	usuarios_roles
SQL	Restricción (Constraint) PK	pk_[tabla]	CONSTRAINT pk_usuarios ...
SQL	Restricción (Constraint) FK	fk_[hija]_[padre]	CONSTRAINT fk_publicaciones_usuarios ...
SQL	Restricción (Constraint) Unique	uq_[tabla]_[columna]	CONSTRAINT uq_usuarios_email ...
SQL	Índice (Index)	idx_[tabla]_[columna]	CREATE INDEX idx_usuarios_nombre ...
SQL	Vista (View)	vw_[proposito]	CREATE VIEW vw_usuarios_activos ...
SQL	Procedimiento Almacenado	sp_[verbo]_[entidad]	CREATE PROCEDURE sp_insertar_usuario ...
SQL	Trigger	tr_[tabla] [momento]_[evento]	tr_usuarios_after_insert